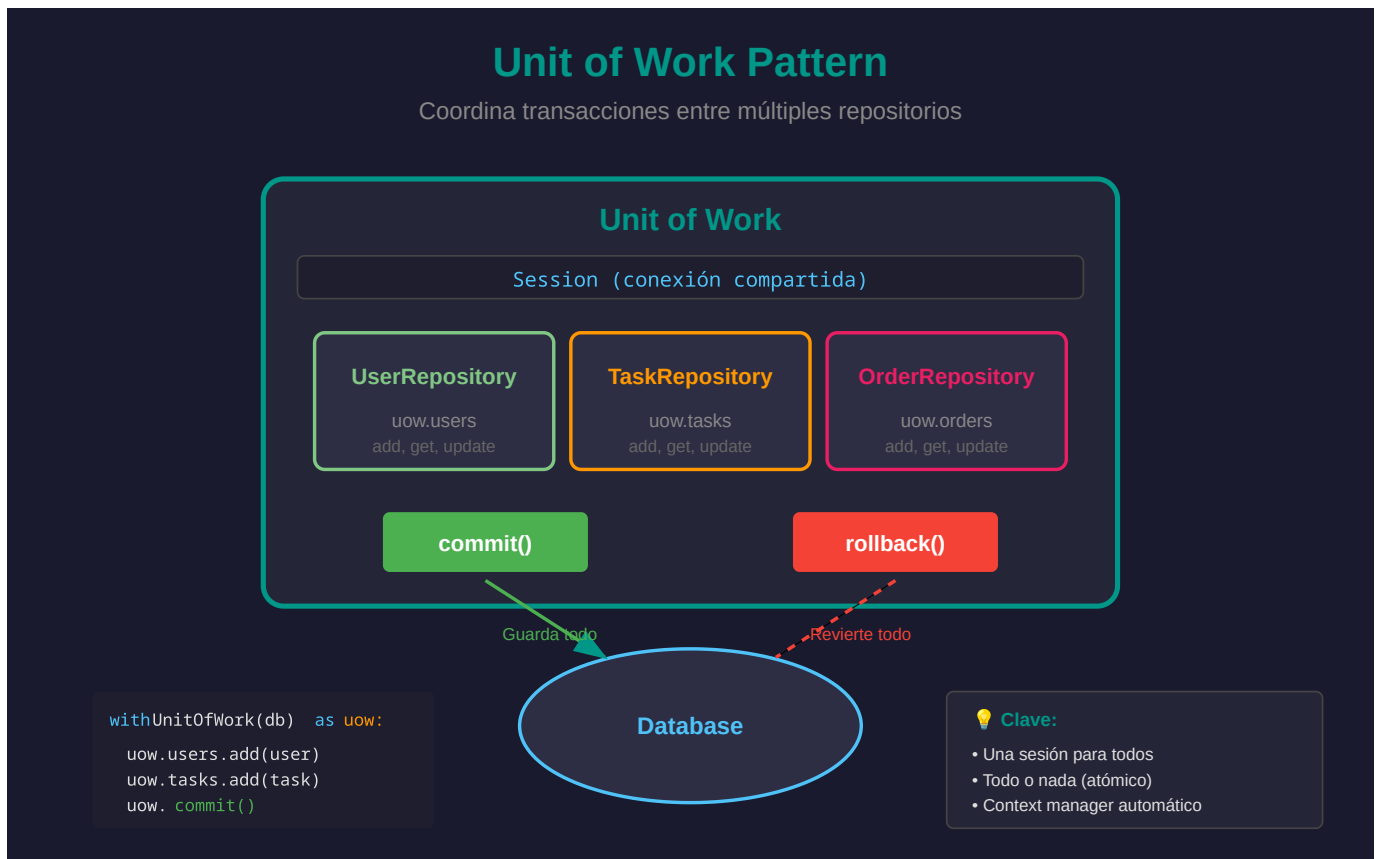


Unit of Work Pattern



Objetivos

- Entender el patrón Unit of Work
- Coordinar transacciones entre múltiples repositorios
- Implementar commit/rollback centralizado
- Integrar con FastAPI dependencies

El Problema: Transacciones Distribuidas

Cuando usas múltiples repositorios, ¿quién hace commit?

```
# ❌ Problema: múltiples commits
class OrderService:
    def create_order(self, data: OrderCreate) -> Order:
        # Crear orden
        order = Order(...)
        self.order_repo.add(order)
        # ¿Commit aquí?

        # Actualizar inventario
        for item in data.items:
            product = self.product_repo.get_by_id(item.product_id)
            product.stock -= item.quantity
            self.product_repo.update(product)
            # ¿Commit aquí?
```

```
# Si falla el inventario, la orden ya está guardada ❌
```

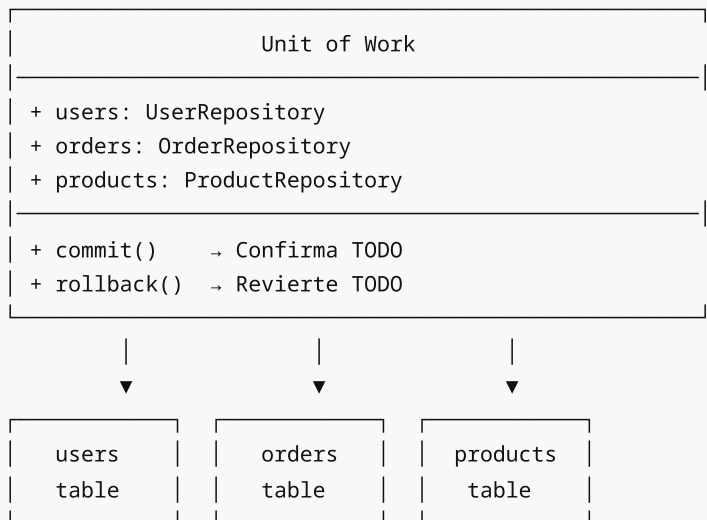
Escenario Problemático

1. Crear orden → commit ✓
2. Actualizar producto 1 → commit ✓
3. Actualizar producto 2 → ❌ ERROR (sin stock)

Resultado: Orden creada, producto 1 actualizado, producto 2 sin cambios
Estado inconsistente de la base de datos

✅ Solución: Unit of Work

El **Unit of Work** (UoW) coordina los cambios de múltiples repositorios en una sola transacción:



🔧 Implementación Básica

```
# repositories/unit_of_work.py
from sqlalchemy.orm import Session

from database import SessionLocal
from repositories import UserRepository, TaskRepository

class UnitOfWork:
    """
    Coordina transacciones entre repositorios.

    Uso:
    with UnitOfWork() as uow:
        user = uow.users.get_by_id(1)
        task = Task(title="New", user_id=user.id)
        uow.tasks.add(task)
        uow.commit()
    """
```

```

def __init__(self, session: Session | None = None):
    # Usa sesión existente o crea nueva
    self._session = session or SessionLocal()
    self._owns_session = session is None

def __enter__(self) -> "UnitOfWork":
    # Crear repositorios con la misma sesión
    self.users = UserRepository(self._session)
    self.tasks = TaskRepository(self._session)
    return self

def __exit__(self, exc_type, exc_val, exc_tb):
    if exc_type is not None:
        # Hubo excepción → rollback
        self.rollback()

    if self._owns_session:
        self._session.close()

def commit(self) -> None:
    """Confirma todos los cambios"""
    self._session.commit()

def rollback(self) -> None:
    """Revierte todos los cambios"""
    self._session.rollback()

@property
def session(self) -> Session:
    """Expone sesión para casos especiales"""
    return self._session

```

Uso con Context Manager

Caso Simple

```

# Crear usuario y task en una transacción
with UnitOfWork() as uow:
    user = User(name="John", email="john@example.com")
    uow.users.add(user)

    task = Task(title="First task", user_id=user.id)
    uow.tasks.add(task)

    uow.commit() # Ambos se guardan o ninguno

```

Manejo de Errores

```

with UnitOfWork() as uow:
    try:
        user = uow.users.get_by_id(1)
        if not user:

```

```

        raise NotFoundError("User not found")

    task = Task(title="New task", user_id=user.id)
    uow.tasks.add(task)

    uow.commit()

except Exception:
    uow.rollback()
    raise

```

Integración con FastAPI

Opción 1: Dependency que crea UoW

```

# dependencies.py
from repositories.unit_of_work import UnitOfWork

def get_uow():
    """Dependency que proporciona Unit of Work"""
    with UnitOfWork() as uow:
        yield uow

# routers/tasks.py
@router.post("/", response_model=TaskResponse)
def create_task(
    task_data: TaskCreate,
    uow: UnitOfWork = Depends(get_uow)
):
    # Verificar usuario
    user = uow.users.get_by_id(task_data.user_id)
    if not user:
        raise HTTPException(404, "User not found")

    # Crear task
    task = Task(**task_data.model_dump())
    uow.tasks.add(task)
    uow.commit()

    return task

```

Opción 2: UoW recibe Session existente

```

# dependencies.py
from database import get_db

def get_uow(db: Session = Depends(get_db)):
    """UoW con sesión de FastAPI"""
    with UnitOfWork(session=db) as uow:
        yield uow

```

UoW en Services

El patrón más limpio es usar UoW dentro de los services:

```
# services/task_service.py
from repositories.unit_of_work import UnitOfWork
from exceptions import NotFoundError

class TaskService:
    """Service que usa Unit of Work internamente"""

    def __init__(self, uow: UnitOfWork):
        self.uow = uow

    def create_task(self, data: TaskCreate) -> Task:
        # Validar usuario existe
        user = self.uow.users.get_by_id(data.user_id)
        if not user:
            raise NotFoundError(f"User {data.user_id} not found")

        # Crear task
        task = Task(**data.model_dump())
        self.uow.tasks.add(task)

        # NO commit aquí - lo hace quien llama al service
        return task

    def complete_task(self, task_id: int) -> Task:
        task = self.uow.tasks.get_by_id(task_id)
        if not task:
            raise NotFoundError(f"Task {task_id} not found")

        task.status = TaskStatus.DONE
        task.completed_at = datetime.utcnow()

        return task

    def assign_task(self, task_id: int, user_id: int) -> Task:
        task = self.uow.tasks.get_by_id(task_id)
        if not task:
            raise NotFoundError(f"Task {task_id} not found")

        user = self.uow.users.get_by_id(user_id)
        if not user:
            raise NotFoundError(f"User {user_id} not found")

        task.user_id = user_id
        return task
```

Uso en Router

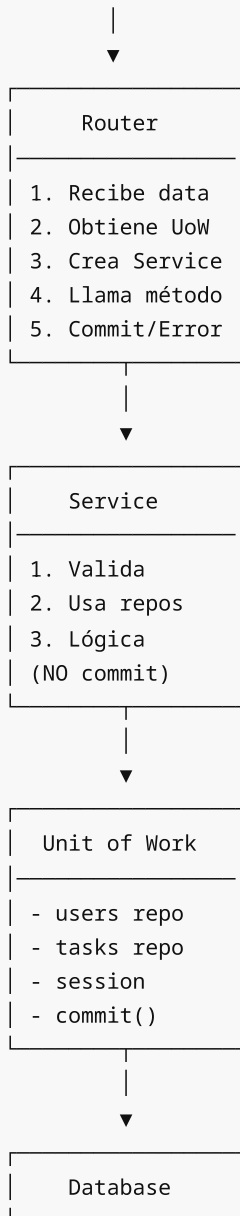
```
# routers/tasks.py
@router.post("/", response_model=TaskResponse)
```

```
def create_task(
    task_data: TaskCreate,
    uow: UnitOfWork = Depends(get_uow)
):
    service = TaskService(uow)

    try:
        task = service.create_task(task_data)
        uow.commit()
        return task
    except NotFoundError as e:
        raise HTTPException(404, str(e))
```

Flujo Completo

Request POST /tasks



⚠ Consideraciones

¿Cuándo Usar UoW?

Situación	¿UoW?
Operación con un solo repositorio	Opcional
Operación con múltiples repositorios	✅ Recomendado
Transacciones que deben ser atómicas	✅ Necesario
CRUD simple	Probablemente no

Errores Comunes

```
# ❌ MAL - commit en cada operación
def create_order(self, data):
    order = Order(...)
    self.uow.orders.add(order)
    self.uow.commit() # ← Commit prematuro

    for item in data.items:
        # Si falla aquí, la orden ya está guardada
        ...

# ✅ BIEN - commit al final
def create_order(self, data):
    order = Order(...)
    self.uow.orders.add(order)

    for item in data.items:
        ...

    # Commit después de todas las operaciones
    self.uow.commit()
```

✅ Checklist

- ☐ Entiendo qué problema resuelve Unit of Work
- ☐ Sé implementar UoW como context manager
- ☐ Puedo integrar UoW con FastAPI dependencies
- ☐ Sé cuándo hacer commit (al final, no en medio)

🔗 Siguiente

Aprenderemos cómo el Repository Pattern **facilita el testing** con mocks y fakes.

→ [05-testing-con-repositories.md](#)